

MIT Poker Bots IAP Report

Team name: hi

Team member: Tina Zhang - zhangtin@mit.edu

Summary

Our pokerbot is a combination of heuristic-based decision-making, opponent modeling, and exploration of game theory optimization. Our pokerbot's architecture consists of four main components: a C++ equity calculation engine for fast Monte Carlo simulations, dynamic programming bet tables, opponent modeling, and a card discard decision algorithm. We also explored Chance Sampling Monte Carlo Counterfactual Regret Minimization (CFR) and Proximal Policy Optimization (PPO)-based reinforcement learning.

Texas Hold'Em Variant Game Rules

The Texas Hold'Em variant game starts with each player with three initial hole cards and a sequential discard phase. After the 2-card flop, players discard one hole card to the board, with the out-of-position player tossing first then the dealer, which creates a final board with six cards. These discarded cards affect hand evaluation, and the 6-card board leads to increased chances of strong hands, which requires tighter postflop standards, such as folding more often and a more conservative betting range. Moreover, the dealer gains a significant information advantage by seeing the out-of-position player's discarded card before tossing their own.

Bot Architecture

1. C++ Equity Engine

We developed a fast hand evaluator in C++, which is compiled as a shared library (.so) for integrating with Python. The C++ engine runs a range of 2,000 to 10,000 Monte Carlo simulations per decision, far outweighing the roughly 50 possible simulations in Python within the time constraints, leading to more accurate equity estimates and thus better 5-card hand decisions. Monte Carlo is a computational technique that uses repeated random sampling to model the probability of different outcomes in complex systems, essentially using chance to find solutions to problems that are too difficult for deterministic calculations.

Our C++ equity engine evaluates all $C(8, 5) = 56$ all possible 5-card combinations and keeps the highest scoring one based on the standard hand rankings, such as straights and flushes. Each simulation is a shuffling/dealing of random cards, evaluating our 5-card hand from the 56 combinations to check, evaluating the opponent's 5-card hand, and comparing. The evaluation function is used to perform efficient comparison for the

hand rankings using a 20 bit left shift, where we encoded the hand type in the high bits, so any straight flush automatically beats any quads, etc. Our fast C++ Monte Carlo simulation also takes into account the amount of time left in the game, adjusting the number of simulations accordingly.

2. Dynamic Programming (DP) Bet Tables

To optimize betting, we precomputed Game Theory Optimization (GTO)-inspired betting tables with the fields hand strength, pot category, and street. We categorized hand strength by very strong (greater than or equal to 68% equity), strong (greater than or equal to 52% equity), medium (greater than or equal to 35% equity), and weak (less than 35% equity). We categorized pot categories as small (less than or equal to 12 chips), medium (less than or equal to 50 chips), and large (more than 50 chips). Finally, we have flop, turn, and river as the streets. The DP bet tables encode bet sizes as pot fractions. For example, a very strong medium river has the pot fraction 0.85, which means we bet 85% of the pot. The table gives a pre-computed reasonable bet size for each situation without calculating it during the game.

Additionally, we have bet tables that encode Minimum Defense Frequency (MDF), a concept in game theory, to control our bot's fold frequency as well as bluff frequencies. MDF tells us how often to call to prevent our opponent from profitably bluffing, computed by the fraction between the pot and the pot plus the bet. The resulting fraction tells us if we fold more than the said percentage of times, then the opponent profits by bluffing every time, thus we need to call at least the said percentage of times. As for bluff frequencies, we have a table mapping each combination of the fields to a fraction signifying us to bluff the said percentage of weak hands. Since we also adjust for the type of opponent we play against, there are adjusts to the DP tables' mapping values. For instance, against tighter players we reduce value bets and increase bluffs, while with opponents that call more, we increase value bets and decrease bluffs.

3. Opponent Modeling Implementation

The bot adjusts its strategy based on what type of opponent it's facing instead of playing the same way against everyone. Throughout each match, we track statistics concerning 1. Voluntary Put In Pot (VPIP), which is how often the opponent puts money in when they don't have to. With a high VPIP, it means the opponent plays lots of hands, even the bad ones, and with a low VPIP, the opponent plays only premium hands and folds most; 2. Preflop raise frequency. With a high frequency, it means the opponent is aggressive and likes to build pots, while a low frequency shows a more passive opponent who calls along; 3. Shove rate, which is how often they go all-in or

near all-in preflop. With a high shove rate, it means the opponent is probably bluffing a lot, and with a low rate, it means they only shove with real hands; 4. Postflop aggression, which tracks whether they bet/raise or check/call after the flop ; 5. Check-raise frequency, which is how often they check, wait for us to bet, and then raise. With a high check-raise frequency, it means our opponent is tricky and setting traps, while with a lower check-raise frequency, it means we have a more straightforward player; and 6. Fold-to-raise rate, which is how often they fold when you raise them. With a high fold-to-raise rate, it means they give up more easily thus we can bluff them, while with a lower fold-to-raise rate, it means they don't fold much so we bluff less.

From these statistics, we categorize our opponents into seven types: "shove bot" (keeps going all-in preflop, either has a very simple strategy or trying to bluff you. We counter these types of opponents by calling with decent hands since they're often bluffing), "maniac" (plays almost every hand and bets or raises constantly, being very aggressive. We counter these types of opponents by letting them bet into us and then calling down with medium strength hands), "calling station" (plays lots of hands but keeps calling and doesn't raise or fold much. We counter these types of opponents by betting big with good hands since they'll pay us and never bluff since they'll call anyway), "trapper" (checks strong hands hoping we bet then raises), "loose aggressive" (plays many hands and bets aggressively but more balanced than a "maniac"), "very tight/tight" (only plays premium hands and folds most of the time. We counter these opponents by stealing their blinds often and fold when they bet big since it's less likely to be bluffing), and "normal" (balanced with no obvious areas to exploit).

4. Discard Decision Algorithm

The discard decision is where we determine which card to discard onto the board. The bot evaluates each option by 1. examining how good the two cards it's keeping, 2. whether discarding the card helps the opponent, and 3. whether it can still improve the hand. When looking how good the two cards we keep, the bot looks for pairs where higher rank pairs are better, suited cards where even better if the board already has cards of that suit, connected/adjacent cards that are close in rank for a potential straight, high cards, and board pairing between one of the kept cards and a board card. Moreover, the bot checks for how much the discarded card improves opponent ranges by completing flush draws or straight draws. We try to throw cards that don't connect well with the board and not throw cards that complete obvious draws. Additionally, we examine our drawing equity with the remaining cards. For instance, if we keep two hearts and there's one heart on the board, we could still make a flush with more

hearts coming. Our discard decision is based on a score made up of a combination of these three components and we pick the discard with the highest score.

Strategies

1. Randomization in Betting

Deterministic betting patterns are predictable and exploitable. We randomize our actions so opponents can't be sure with what we have. With strong hands (greater than or equal to 70% equity), we randomize 75% of the time a standard bet of 55% of the pot, 15% of the time a check to protect our checking range, and 10% of the time an overbet of 105% of the pot to look like we're bluffing. We check sometimes even with strong hands to become less predictable for our opponents. Additionally, with very strong hands (greater than or equal to 90% equity), we randomize 70% of the time a standard value bet of 75% of the pot, 15% of the time an overbet of 110% of the pot, and 15% of the time a small bet of 40% of the pot to look weak to provoke the opponent to raise us and then re-raise. We bet different amounts so opponents can't tell if a big bet is a true very strong hand or a bluff.

2. Defensive Mechanisms

We implemented defensive mechanisms against two main scenarios. One is when we're in the big blind where we already put in two chips and then the opponent makes a small raise to about 4-6 chips, and we need to put in 2-4 more chips to continue. If we fold too much here, opponents will raise every hand because it's very profitable. Therefore, against small raises of less than 10 chips, we end up always defending with any pair, any suited cards, and any J or higher rank cards. This makes it difficult for opponents to raise with bad hands and expect us to fold. The other scenario is when the opponent makes a huge raise or goes all-in preflop. Whether we call depends on how often our opponent does this. Against tight opponents with low aggression rates (calculated above in our opponent modeling implementation), since they rarely bluff, we call with only premium hands like QQ+ and AK. Against moderate aggression, we call with strong hands such as 99+ and AQ+. Against high aggression, we call with decent hands such as 88+ and AJ+. Finally, against very high aggression, it's likely they bluff a lot, so we have a wide call range.

3. Progress-based Shift in Playing Style

In poker, if we're winning big, we don't need to take risks. If we're losing badly, we end up risking more to catch up. Our bot automatically shifts our play style according to how the match is going. We have five modes characterizing the state of the match. When we're losing badly and running out of time, we increase bet sizing by

15% trying to gamble to catch up. When we're losing or on a bad streak, we increase bet sizing by 8% to push harder. When it's a close match, we use the standard bet sizing. When we're winning a little, our bet sizing is slightly tight with a 10% smaller bet to protect our small lead. Lastly, when we're winning big, we also have a 10% smaller bet.

Other Explored Approaches

1. Chance Sampling Monte Carlo Counterfactual Regret Minimization (CFR)

CFR is an iterative algorithm used to solve imperfect information games like poker by converging to a Nash equilibrium (GTO). It works by simulating millions of hands, calculating "regret" for actions not taken, and adjusting future strategies to minimize regret at each decision point (information set). We implemented Monte Carlo CFR with external sampling to learn Nash equilibrium strategies. Info set keys encoded street, hand bucket, pot category, position, and action history. We ran training on a 32-core Google Cloud VM achieving ~1.4 million iterations over 48 hours, with around 5,000-6,000 information sets, but time constraints prevented full convergence for tournament deployment.

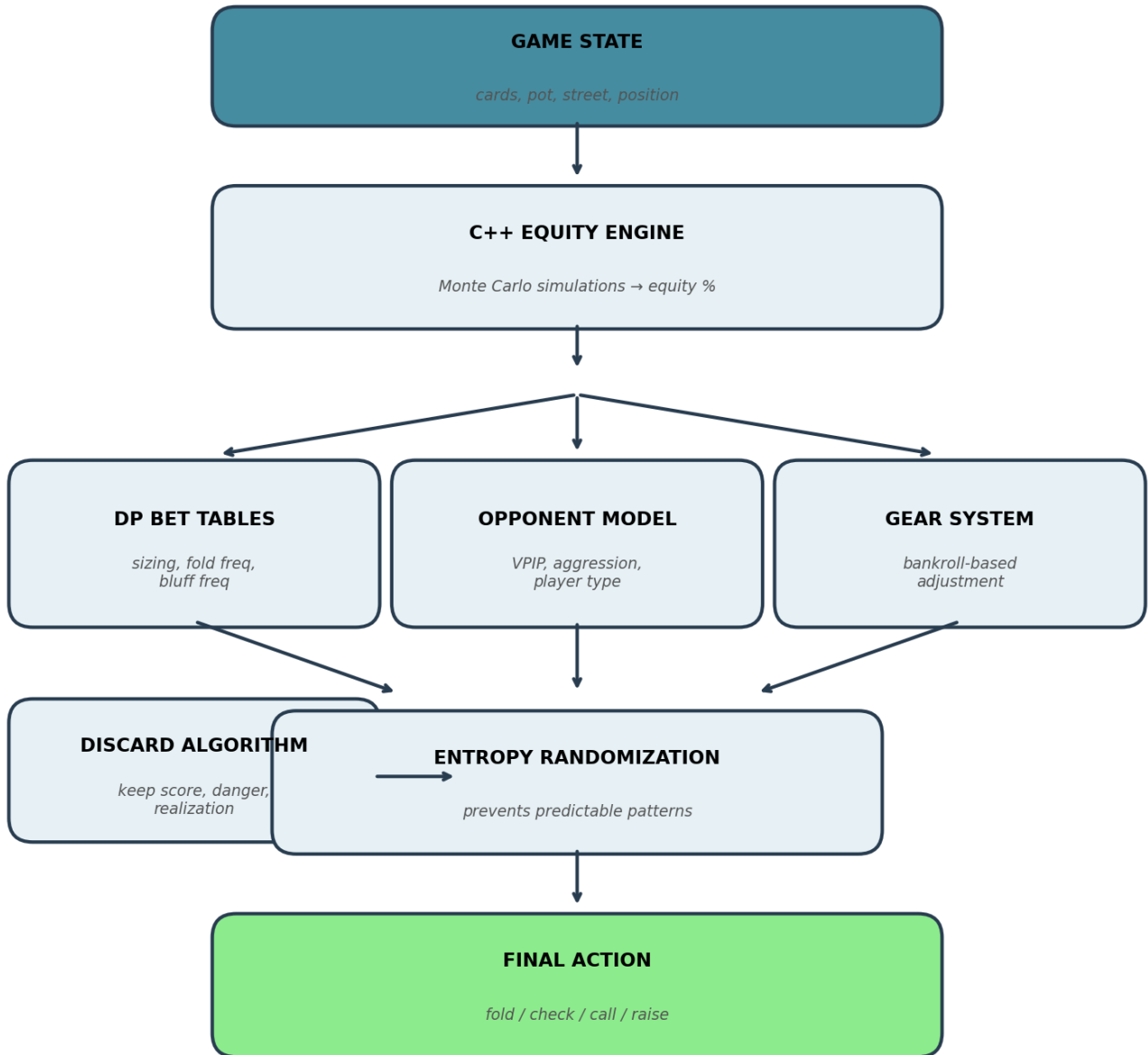
2. Proximal Policy Optimization (PPO)-based Reinforcement Learning (RL)

Proximal Policy Optimization (PPO) is a popular on-policy reinforcement learning (RL) algorithm designed to train agents to maximize cumulative rewards by optimizing a policy network (actor) and a value network (critic). We explored PPO-based deep RL with self-play on GPU (GH100). The agent used 64-dimensional observations and separate action heads for betting (7 actions) and discards (3 options). We ended up training against a heuristic opponent but overall performance was worse than the heuristic bot.

Takeaways and Conclusion

The 60-second game clock taught us the importance of efficient computation. By using the C++ equity engine, we found a much faster way to perform Monte Carlo sampling. We also learned to effectively use dynamic programming for GTO-inspired quick betting table look-ups for fast computation. Moreover, adding noise to betting decisions such as bet size and bluffs helps to prevent opponents from learning patterns. In addition, comprehensive opponent modeling is key to understanding/looking to exploit them with predetermined counter attacks/actions against their actions. We also got to explore CFR and RL as potentially stronger approaches, but implementation and time and compute constraints stood out. Lastly, the iterative process of analyzing game logs with opponents, identifying areas to improve upon, and implementing fixes targeted to these were essential for a stronger bot.

Bot Architecture (v7.7)



Betting decisions use all components; Discard decisions use dedicated algorithm